Lab 05 NSG segmentation and flow diagnosis

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a flow decision table with least-privilege rules. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use py -3 if python3 is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- mock-data.json: synthetic input with no real personal data or credentials.
- analyse.py: runnable analysis for this lab only.
- expected-results.json: expected baseline and source checksum.
- ai-review-prompt.txt: evidence-constrained AI review prompt.
- evidence-template.md: your deliverable template.
- INSTRUCTIONS.md and INSTRUCTIONS.pdf: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open analyse.py in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open results.json. The expected decisions in output order are: ALLOW, DENY, ALLOW. The source_sha256 binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-
results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

NSG rule precedence

Evaluate matching rules by ascending priority; the first match decides a new flow.
Contract: priority, source, destination, port, access
Accepted case: Priority 100 allows web to app TCP 443.
Counterexample: Priority 90 denies the same flow before priority 100.
Diagnosis: Inspect earlier matching rules and both subnet/NIC associations.
Evidence: Record the five-tuple, matched rule and direction.

ASG membership

An application security group groups NICs so rules can express workload roles.
Contract: source_asg=web, destination_asg=app, port=443
Accepted case: The app NIC belongs to the app group used by the rule.
Counterexample: A recreated NIC was never added to the app group.
Diagnosis: Repair membership after checking the actual NIC identity; do not broaden source to Any.
Evidence: Export NIC membership and the effective rule result.

Flow verification

A permitted packet requires matching route, filter and listening service state.
Contract: src_ip, dst_ip, dst_port, direction, protocol
Accepted case: TCP 443 is allowed and the application listens on 443.
Counterexample: NSG allows 443 but the service is stopped.
Diagnosis: Separate route, NSG, host firewall and process failures; IP flow verify alone is not an end-to-end test.
Evidence: Keep filter result plus an application connection test.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change flow-1 port to 8443. It must be denied. Explain the exact rule needed if the application legitimately moves to 8443; do not allow every port.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.
For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure C Build and verify a segmented network

1. In rg-itsec-<initials>, create Virtual network vnet-itsec-<initials> with address space 10.10.0.0/16 in the assigned region.
2. Add subnets web (10.10.1.0/24), app (10.10.2.0/24) and data (10.10.3.0/24). Keep any service-specific reserved subnet separate.
3. Create Application security groups asg-web-<initials>, asg-app-<initials> and asg-data-<initials>. Have the trainer provision or allocate the tier VMs in this new VNet and the corresponding web/app/data subnets. Add only those NICs to the matching ASG; all NICs in an ASG must share a VNet. Record NIC and subnet IDs. If no suitable VM is allocated, record the live flow test as unperformed. Empty ASGs cannot demonstrate a working traffic path.
4. Create nsg-app-<initials> and associate it with the app subnet. In Inbound security rules add priority 100, TCP, source ASG web, destination ASG app, destination port 443, Allow. Name it Allow-Web-App-443.
5. Add priority 200, source VirtualNetwork, destination Any, any port/protocol, Deny for the intended isolation exercise only. This explicit restriction prevents Azure’s default AllowVNetInBound from being mistaken for deny-all. Confirm no shared services depend on this subnet before saving.

6. Create the data-tier NSG and an inbound rule priority 100 allowing TCP 1433 from ASG app to ASG data, followed by the agreed explicit deny policy. Associate it with the data subnet. Record the rule JSON or screenshots.
7. Open Network Watcher > IP flow verify. Select an approved VM/NIC, inbound direction and a real source/destination/port tuple. Test the intended allow and deny cases and record the matched rule. A VM/NIC must exist for this operational test; a diagram is not a substitute.
8. Inspect NIC effective security rules and routes. If the filter allows traffic but the application fails, inspect the host firewall and listening service through the trainer-approved access path. Do not broaden ports to bypass the diagnosis.
9. Compare live behaviour with the simplified Python fixture: Azure also has default rules, associations and stateful connection handling absent from the script. Retain configuration, allow/deny evidence and limitations. Remove only lab-created network resources after all later labs are complete.
For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a flow decision table with least-privilege rules with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.